Exploring and Learning on Web Graphs: Introduction and advanced community notions

Oana Balalau

Inria, Ecole Polytechnique
oana.balalau@inria.fr

13/03/2026

We thank Ioana Manolescu for part of the material.

Class Overview

Class Overview

Lectures:

- How to represent data: the expressive power of different representations
- Social networks: advanced notions on community detection
- Entity matching between different datasets
- Natural language processing to extract and classify information from text sources
- An introduction to statistics in social research

The first four sessions are dedicated to the presentation of concepts and tools, while in the following sessions, you will carry out projects in pairs with the help of the tutor. You will have complete freedom to choose the topic of your project.
Prerequisites: Python
Language: The material is in English, but the course can be in English or French, depending on the students.

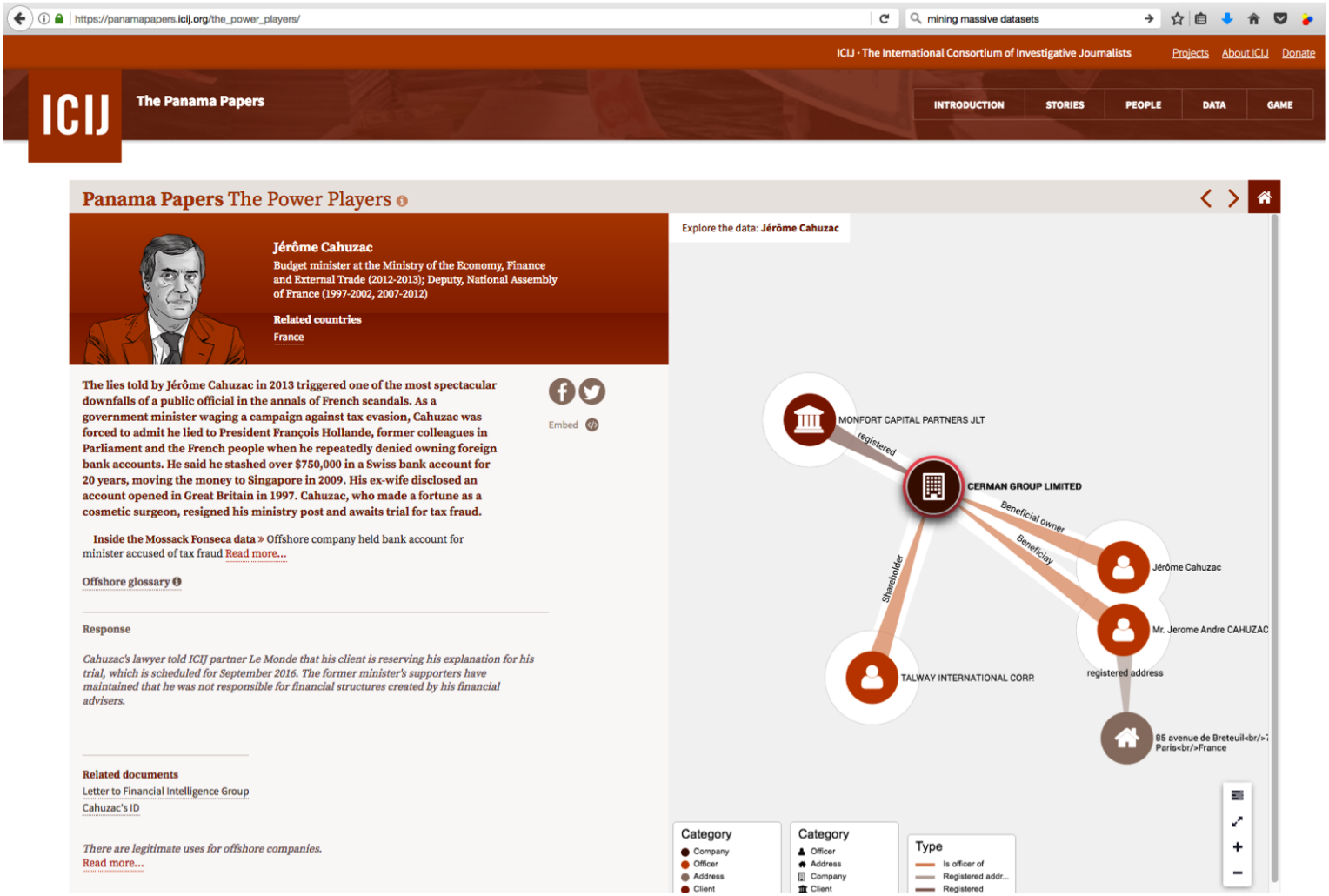
What data can we use?

Open Government Data (OGD) *is a philosophy - and increasingly a set of policies - that promotes transparency, accountability and value creation by making government data available to all. Public bodies produce and commission huge quantities of data and information. By making their datasets available, public institutions become more transparent and accountable to citizens.*

`https://www.data.gouv.fr/fr/`
`https://data.europa.eu/en`
`https://data.worldbank.org/`
`https://www.who.int/data/gho/`
You can of course find data on other sites!

What can **we** do with the data?

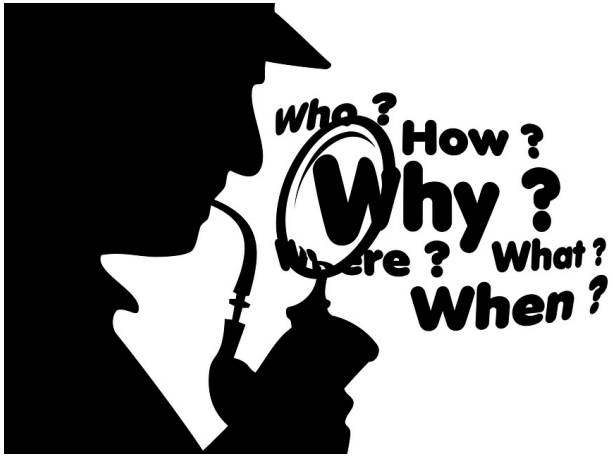
Journalists have analysed **data leaks**, such as the Panama Papers to find **conflicts of interest**.



What can **we** do with the data?

In previous years, we had projects on:

- analysing political data, such as voting trends on laws in the European Parliament
- how climate change is discussed on social media
- analysing differences between French and Chinese cultures via movies datasets
- analysing videogames and gamers using Reddit and Twitch data
- finding patterns in scientific paper citations
- and many other cool ideas!



What can **we** do with the data?

Computational social science: investigate human activity and relationships using large-scale demographic, behavioral and network data. The goal is to investigate and find solutions to **societal challenges** through the lenses of the legal, economic, socio-cultural and political contexts.

A lot of very interesting research results are being published! For example:
`https://www.nature.com/collections/cadaddgige`
`https://dblp.org/db/conf/icwsm/index.html`

Data on the Web

Structured, Semi-structured and Unstructured

Structured, semi-structured, unstructured: terms used to classify data based on its format and schema rules.
Structured data characteristics:

- Identifiable structure built on a data model
- Organization in rows and columns, with fixed fields
- Storage in a tabular form, such as in databases
- Meaning of the data is clear, hence users and computers can access and query it
- Data points in the same class share the same attributes – e.g. a telephone number field will always be numerical and have a set number of digits

Structured, Semi-structured and Unstructured

Unstructured data characteristics:

- No identifiable structure/organization, data model
- Data points can vary widely – the same dataset could contain video, audio, image and text files; or just the same type of data but with very different properties.
- It cannot be easily analyzed for meaning or trends without adding additional information on its structure.

Semi-structured data characteristics:

- no predefined data model
- has however metadata - data about data such as tags, but not as complete as structured data (e.g. it covers only part of the data, or it allows ambiguities)
- the metadata facilitates searching and analysis.

Semi-structured data was fulled by Internet communications, where only text or structured data was no longer viable.

File Types

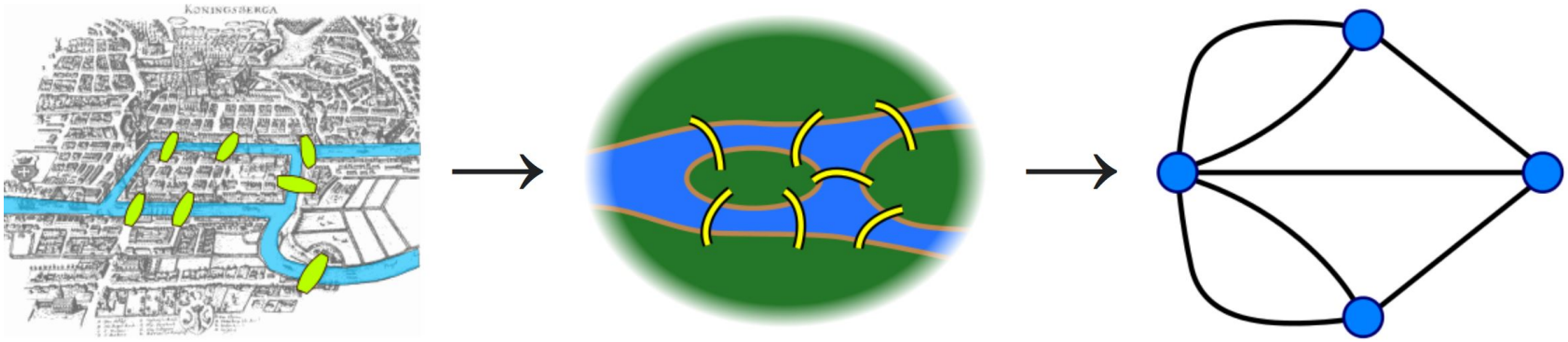
File types mentioned using their extension:

- **Structured data**: file format associated with databases: DB (general database file), SQLITE (SQLite database file), MYD (MYSQL database file), etc.
- **Unstructured data**: TXT, images and videos file types
- **Semi-structured data**: CSV, JSON, HTML, XML

Note that CSV can store structured data, but this is not enforced by the file format (e.g headers are not necessary present and rows can have different numbers of columns).

Advantages of representing data as graphs

Certain data representations can help us easily gain knowledge about the data.
Seven Bridges of Königsberg, Euler, 1736



Graphs are **mathematical objects**, hence we can prove that they have certain properties.

The notions of an edge and node are very versatile: simple graphs, but also weighted graphs, hypergraphs, multigraphs, and data graphs.

Small worlds

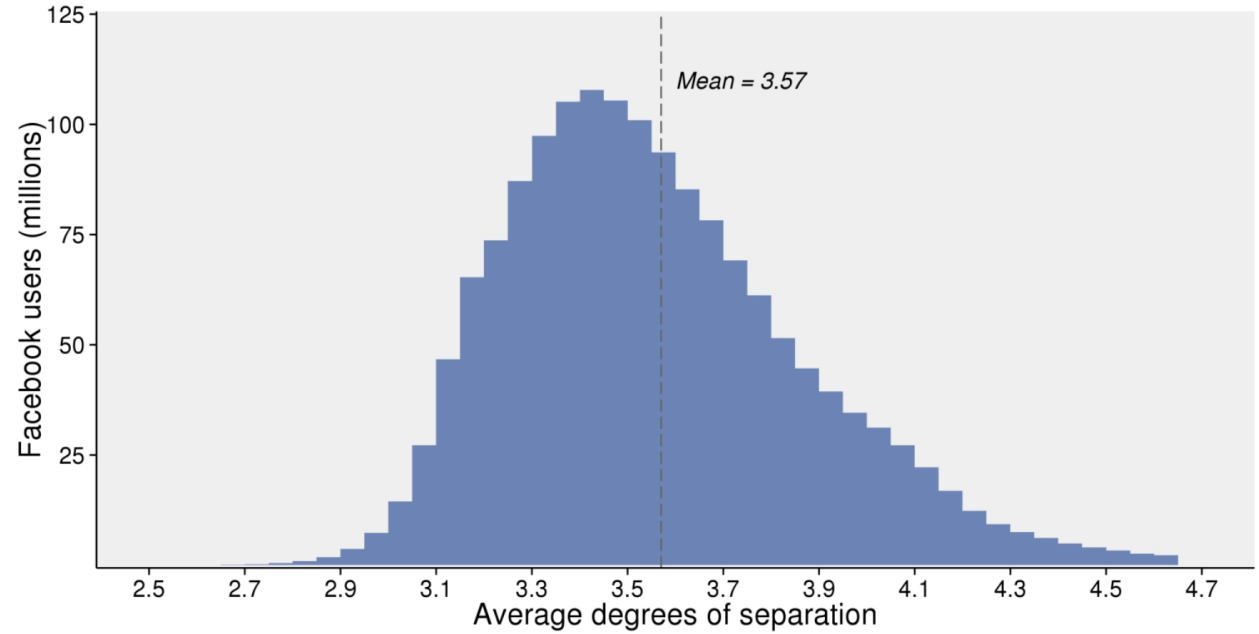
Experiment made by experimental psychologist Stanley Milgram in the 1960s

1. Sent 96 packages to randomly chosen people from Omaha, Nebraska, telling them each package includes a **passport** and must reach a **target person** living in the US or abroad, unknown to the sender, but whose name, occupation and city were given.
2. Each person can only send the package to someone known **on a first-name basis**, who can only send the package to a similarly well-known acquaintance, to reach the destination.
3. On average, each package traveled along a path of length **5.9**.

Conclusion that "short paths tend to be short" widely accepted/confirmed, despite limitations/flaws in Milgram's early study.

Small modern worlds

On Facebook, the average distance between two users is 4.57.[1].



[1] https://research.fb.com/blog/2016/02/three-and-a-half-degrees-of-separation/

Graphs and their basic notions

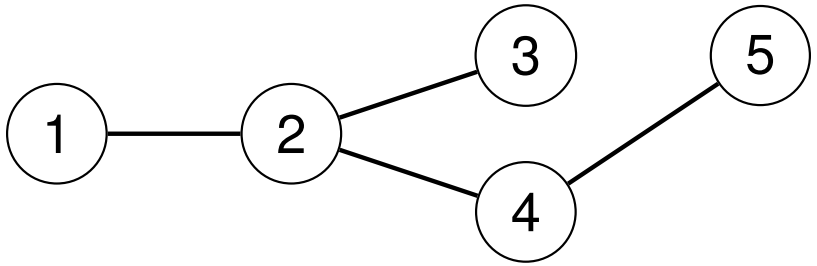
Undirected graphs

A graph consists of **nodes (vertices)** and **edges**.

$$G = (N, E), N = \{n_1, n_2, \ldots, n_{|N|}\}, E = \{e_1, e_2, \ldots, e_{|E|}\}, E \subseteq (N \times N)$$

Nodes typically denote entities (things) while edges denote (binary) relationships between nodes

Example
N is a set of people, E states which people know each other.



The relation E is **symetric** $\Leftrightarrow$ the graph is **undirected**.

The **degree** of a node is the number of its adjacent edges.

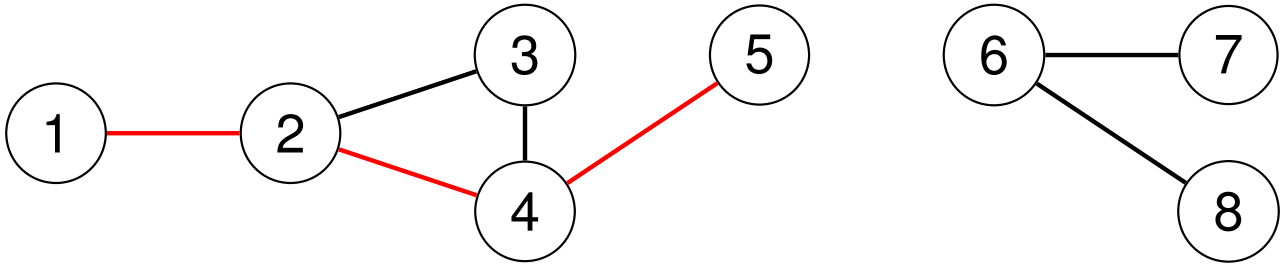
Graph paths

$G = (N, E), E \subseteq (N \times N)$

A **path** p of **length** k is a sequence of edges $e_1, e_2, \ldots, e_k$ such that $\forall 1 \leq i < k$, $e_k = (n_1^k, n_2^k), e_{k+1} = (n_1^{k+1}, n_2^{k+1})$, we have $n_2^k = n_1^{k+1}$.

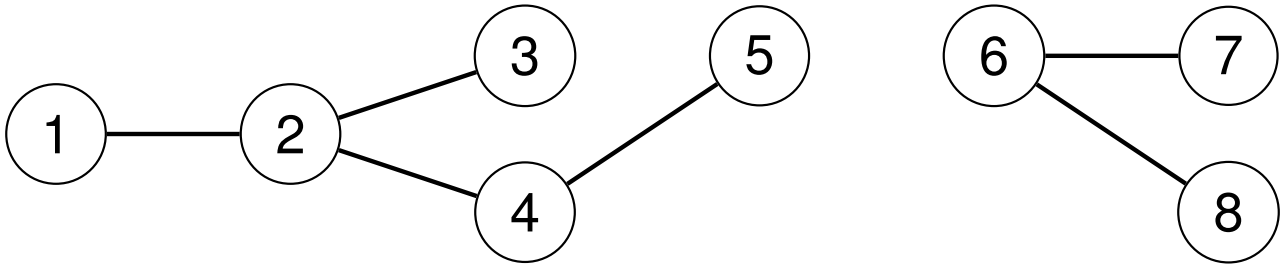
p is also denoted by the sequence of nodes $n_1.n_2.\ldots.n_{k+1}$.

Node n_2 is **reachable from** node n_1 if there exists a path starting in n_1 and ending in n_2.



$2, 3, 4, 5$ are reachable from 1; $6, 7, 8$ are not.

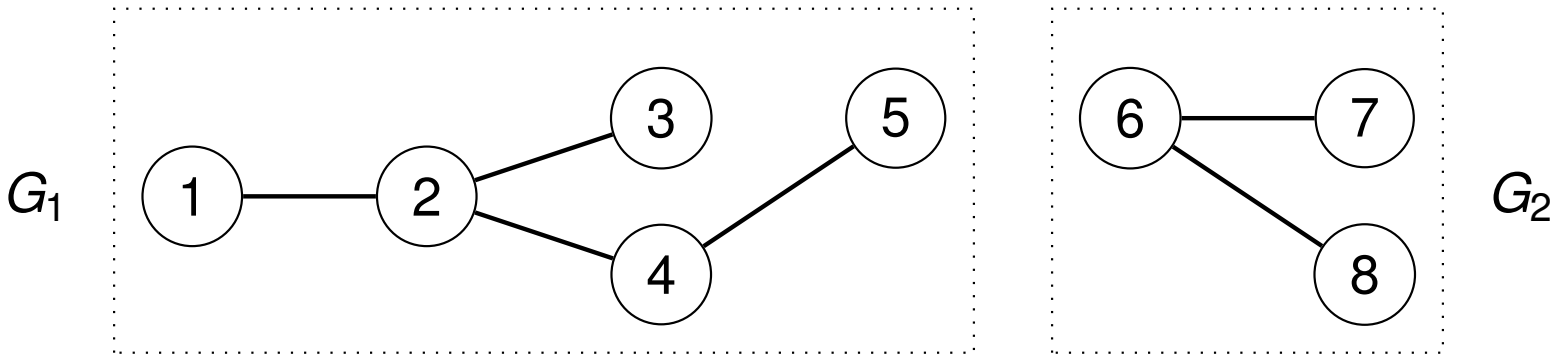
Graphs connectivity



A **connected component** of the graph $G = (N, E)$ is a pair $G_1 = (N_1 \subseteq N, E_1 \subseteq E)$ such that

1. $\forall n_1, n_2 \in N_1$, n_2 is reachable from n_1
   - N_1 nodes are all reachable from each other
2. $\forall n_1 \in N_1, n_2 \in N$ s. t. n_2 is reachable from n_1, we have $n_2 \in N_1$
   - N_1 contains all the nodes reachable from any of its nodes

Graph connectivity



A **connected component** of the graph $G = (N, E)$ is a pair $G_1 = (N_1 \subseteq N, E_1 \subseteq E)$ such that

1. $\forall n_1, n_2 \in N_1$, n_2 is reachable from n_1
   - N_1 nodes are all reachable from each other
2. $\forall n_1 \in N_1, n_2 \in N$ s. t. n_2 is reachable from n_1, we have $n_2 \in N_1$
   - N_1 contains all the nodes reachable from any of its nodes

Directed graphs

$G = (N, E)$ but $e = (n_1, n_2) \in E \not\Rightarrow e' = (n_2, n_1) \in E$
Directed graphs correspond to asymmetric relations between nodes. E.g.: Twitter "follows", being someone's manager, professor, biological child; a Web page with a link to another page

An <u>un</u>directed graph $G = (N, E)$ can be seen as equivalent to a directed one $G' = (N, E')$, where for every $e = (n_1, n_2) \in E$, $e' = (n_1, n_2)$ and $e'' = (n_2, n_1)$ are in E'.

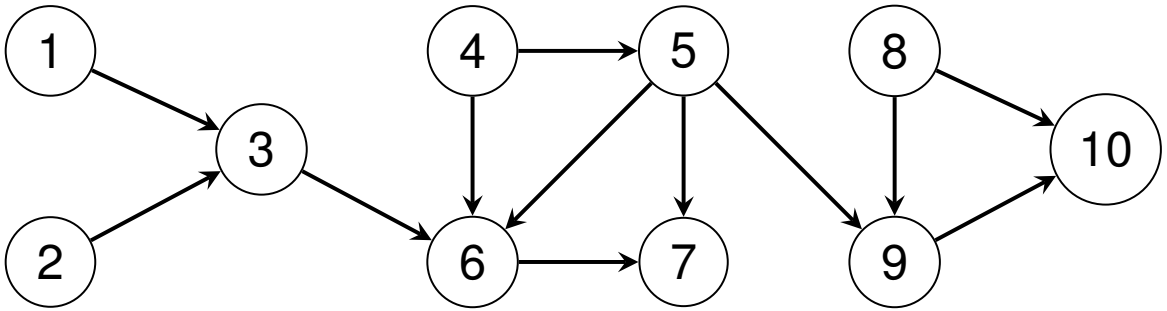
Directed graphs: what changes?

An edge has a **source** and a **target**.

The number of edges with source is n is the **fan-out (out-degree)** of n.

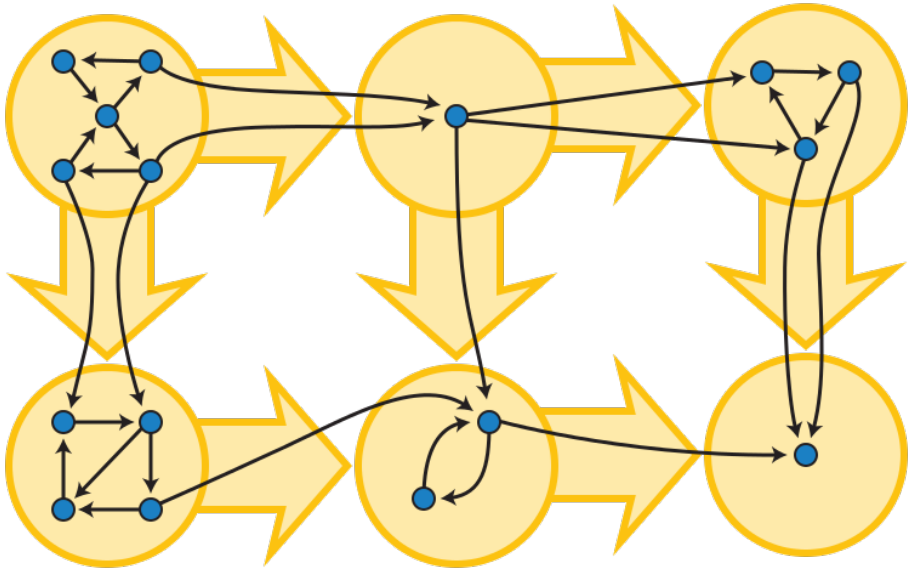
The number of edges with target is n is the **fan-in (in-degree)** of n.

Reachability is no longer symmetric, because a path follows only forward edges:



Strongly connected components

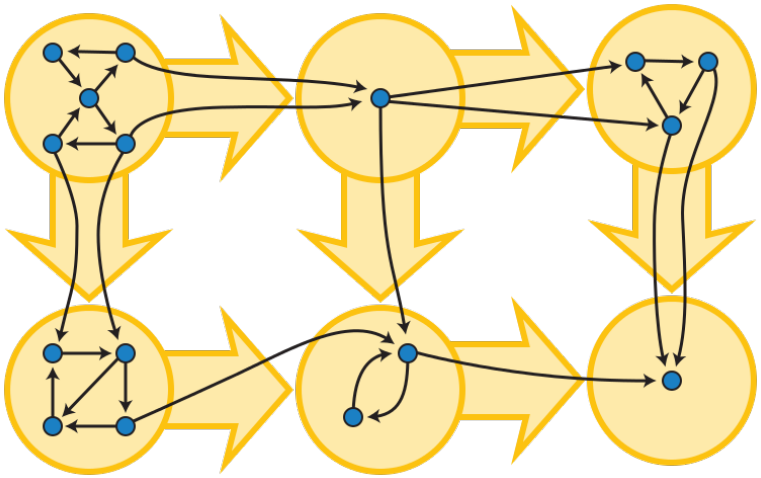
Given $G = (N, E)$, the graph $G_1 = (N_1 \subseteq N, E_1 \subseteq N_1 \times N_1)$ such that each N_1 node is reachable from all the others forms a **strongly connected component** of G.
From $G = (N, E)$ one can obtain $G_{SCC} = (N_{SCC}, E_{SCC})$, the graph of its strongly connected components:



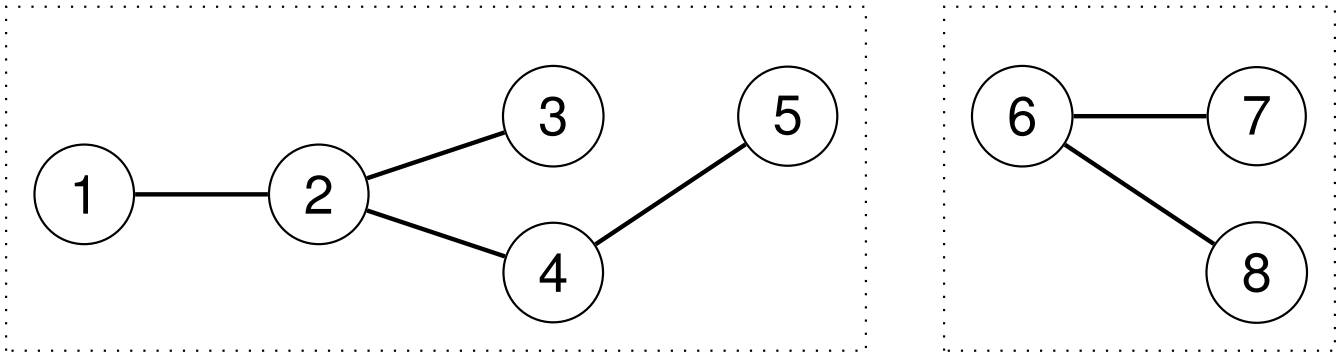
There is a **linear algorithm for computing strongly connected components** based on depth first search.

Strongly connected vs. connected components

Directed graph: there are connections across components



Undirected graph: by definition, components are isolated. Typically, each component is processed independently.



Importance (score) of a graph node

Measuring the importance of a node in a graph

Importance (or **score**, or **centrality**) can be defined in many different ways, each one useful in specific applications.

1. Degree. A node with a high degree is more likely to be reached by some propagation in the network. Aka **degree centrality**

- A node with a high in-degree can be seen as “popular” (for “follows”)
- A node with a high out-degree can be seen as “curious”

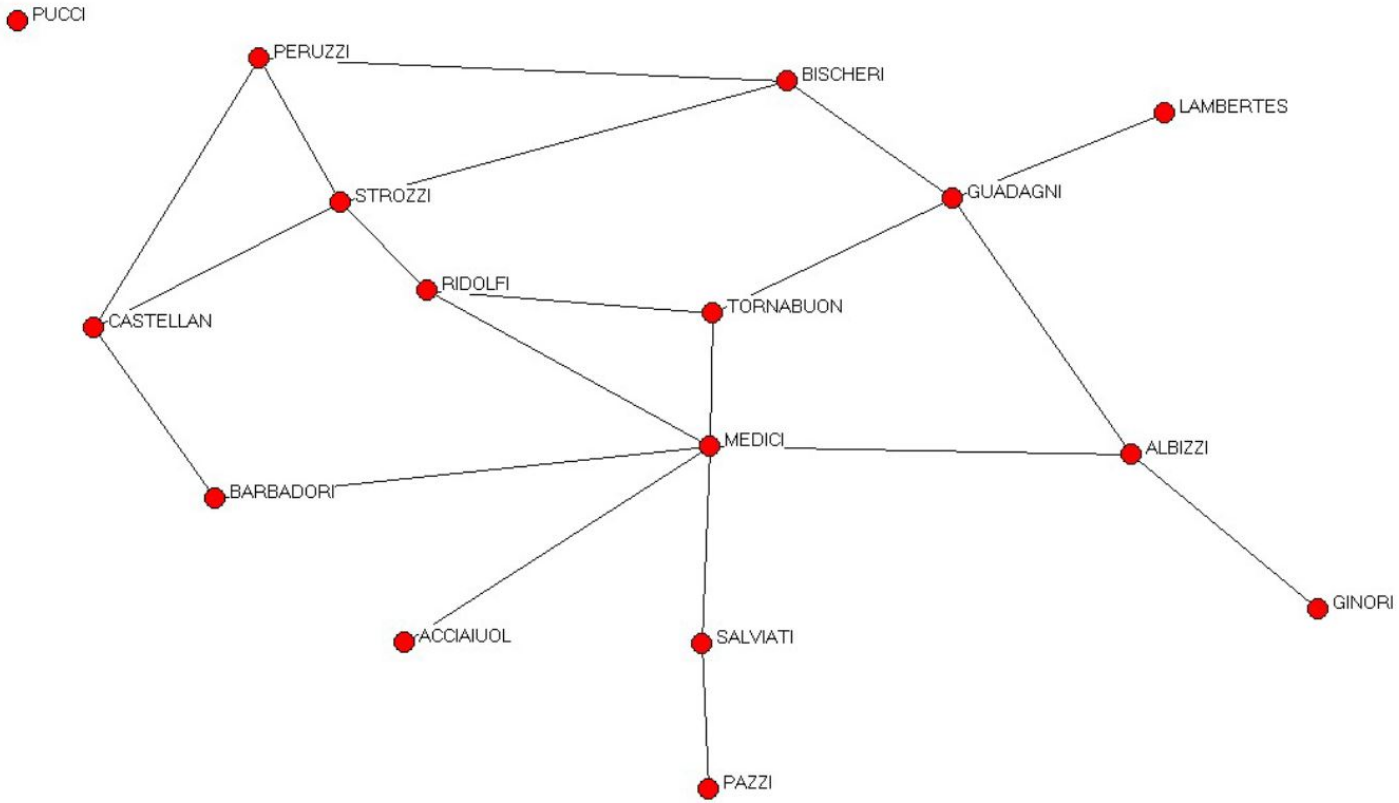
Betweenness centrality

2. Betweenness centrality of the node $n \in G$ reflects how many of the shortest paths in G pass through n.
For $n_i, n_j \in N$, let $n_{i,j}$ be the number of distinct shortest paths from i to j.
For $n_i, n_j, n_k \in N$, let $n^k_{i,j}$ be the number of shortest paths from n_i to n_j passing through n_k.
The betweenness centrality of n_k is:

$$b_k = \sum_{i,j \text{ such that } n_{i,j}>0} (n^k_{i,j} / n_{i,j})$$

The values could be normalized by dividing by $(|N| - 1)(|N| - 2)/2$.
Maximum for: center of a star, but also bridge. Minimum for: leaf
Nodes with high betweeness **control communication** in a network.

15th Century Marriages in Florence



Medici vs Guadagni: degree 6 vs 4, betweeness .522 vs .255[2]

[2] https://web.stanford.edu/~jacksonm/netbook.pdf

Eigenvector centrality

3. Eigenvector centrality is based on the principle:

A node is important if it is linked to by other important nodes [Leo41, See49]

Let $A = (a_{i,j})$, $1 \leq i, j \leq |N|$ be the **adjacency matrix** of G:

$$a_{i,j} = 1 \text{ if } (n_i, n_j) \in G \text{ and } 0 \text{ otherwise.}$$

For each n_i, the (left) **eigenvector centrality** of n_i, denoted ev_i, satisfies:

$$ev_i = \frac{1}{\lambda} \sum_k ev_k \, a_{k,i} \text{ for a given } \lambda > 0$$

"<u>The edge from k to i transfers (adds) the centrality of n_k to n_i</u>".

$$ev = \frac{1}{\lambda} ev \, A$$

Eigenvector centrality

Chose λ as the eigenvalue of A with the highest absolute value.
Perron-Frobenius theorem: if A is irreducible (if G is strongly connected), then the eigenvector solution ev exists, is **unique** and **positive**.

Computing ev by the **power method**

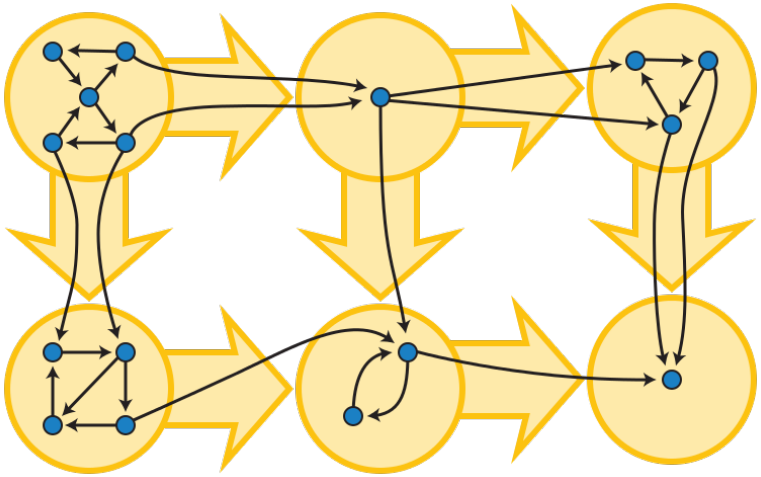
1. Start from $ev_i^{(0)} = 1/|N|$ for every i. Set $k = 1$.
2. Repeat
   1. compute $ev^{(k)} = ev^{(k-1)}A$;
   2. normalize $ev^{(k)}$
   3. $k \leftarrow k + 1$

   until $ev^{(k)}$ is sufficiently close to $ev^{(k-1)}$.

Limitations of eigenvector centrality

Eigenvector centrality works well for undirected graphs.

In directed graphs, nodes in **strongly connected components** or in **out-components of these** have high eigenvector score.
Other nodes can have very low or 0 eigenvector scores!



Limitations of eigenvector centrality

In a directed graph that is not strongly connected:

Perron-Frobenius assumptions are not valid, than the solution of the eigenvector problem is not necessarily unique and positive.

Can we do something better?

Google's original PageRank centrality

A node is important if it is linked from other important nodes <u>with low out-degree</u> or if it is highly linked.

5. PageRank of n_i: $pr_i = \alpha \sum_k \frac{a_{k,i}}{d_k} pr_k + (1-\alpha) pr_0$ [BP98]

where: $d_k = \begin{cases} \text{out-degree of } n_k & \text{if } n_k \text{ has outgoing edges} \\ 1 & \text{otherwise.} \end{cases}$

α is a damping factor, pr_0 is initial importance score ($1/|N|$, $1/|N|$ ...).

The graph becomes strongly connected via the $(1-\alpha)$ transitions.

6. Personalized PageRank. For a node n_i the initial pr_0 is 1 in position i and 0 otherwise.

From nodes to communities

Calculating centrality measures on modern social media platforms (like Twitter/X or Facebook) often reveals highly polarized networks.

In these networks, users tend to link to, share, and amplify other users who hold similar views.

Because users cluster so tightly around central figures within their own "echo chambers", we need specific algorithmic tools to identify and separate these distinct groups.

Definition of community

Semantic definition: a community should contain members that share a similarity between themselves, and they do not share it with entities outside the community. e.g. a class, the people having a certain job.

Representation in a graph: Given a graph where nodes represent entities or concepts and edges represent semantic links between them (e.g. friendship), find subsets of nodes $C1, \ldots, Ck$ corresponding to communities.
The problem is called non-overlapping or overlapping community detection depending on whether the Ci 's are disjoint or not.
Depending on the task, we might find a **partition** of V or a **subset**.

Community detection based on cliques

Cliques and graph density

A graph is **complete** (**clique**) if for every $n_1, n_2 \in N$, $e = (n_1, n_2) \in E$. The graph density is the ratio of the number of edges E with respect to the maximum possible edges.

Undirected graphs: $D = \frac{2|E|}{|N|(|N|-1)}$, maximum number of edges in G can be $C^2_{|N|}$.

Directed graphs: $D = \frac{|E|}{|N|(|N|-1)}$, maximum number of edges in G can be $A^2_{|N|}$.

Clique percolation

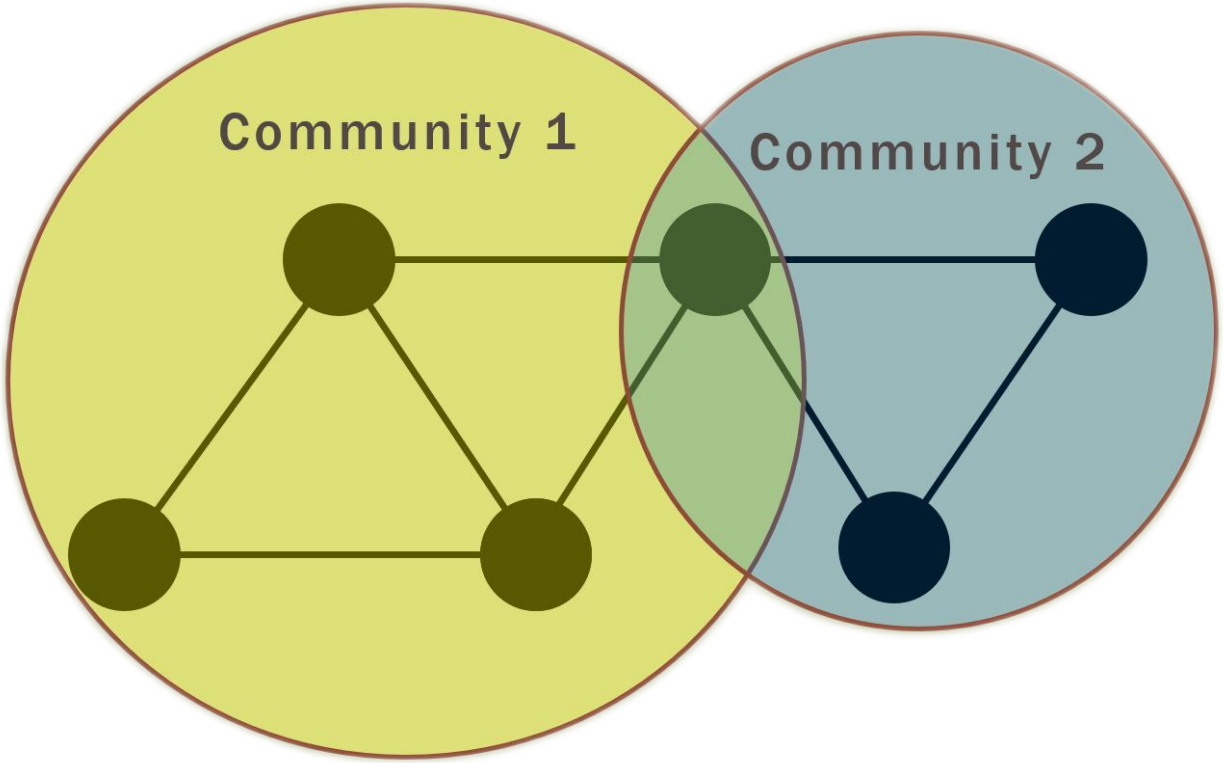
Clique percolation is a method to find **overlapping communities**.

Based on concept:
- internal edges of community likely to form cliques
- intercommunity edges unlikely to form cliques

Adjacent k-cliques:
two k-cliques are adjacent when they share k-1 nodes.

Clique percolation

k-clique community : Union of all k-cliques that can be reached from each other through a series of adjacent k-cliques



Clique percolation: Algorithm

Clique percolation algorithm:

- Locate maximal cliques
- Find k-clique communities via clique adjancency matrix

A **maximal clique** is a clique to which no more vertices can be added.

The **clique problem** is the computational problem of finding a **maximum clique**, or all cliques, in a given graph. It is NP-hard.

The maximum clique is also a maximal clique, hence the clique percolation algorithm has the worst case runtime complexity exponential in the number of nodes.

Clique percolation: Algorithm

Algorithm for finding k cliques: We start by finding k cliques in decreasing order of their size. The maximum size of a clique is <= maximum degree + 1. We iterate to find all clique of size k and check if they are not included in a bigger clique.

1. Let n_i be the first node in decreasing order of the nodes' degree.
2. We determine the **k cliques that contain the node** n_i.
3. We then remove the node from the graph and continue the algorithm.

How we determine the maximal clique containing a given node?

Clique percolation: Algorithm

Algorithm for maximal cliques that contain n_i. Two sets of nodes:

- **A** contains a clique (initially n_i)
- **B** contains nodes that are connected to each node in A, but not necessarily between themselves (initially neigh. of n_i)

We proceed as follows:

1. We set an order over the nodes in the neigh. of n_i and we move them from B to A.
2. When a node n_j is moved, we remove from B all the nodes not connected to it. -> this is done recursively.
3. When B is empty, if the clique in A is not included in another clique we found, we mark it and we explore another combination of the neighbours of n_i.

Clique percolation: Algorithm

Algorithm for finding k-clique communities:

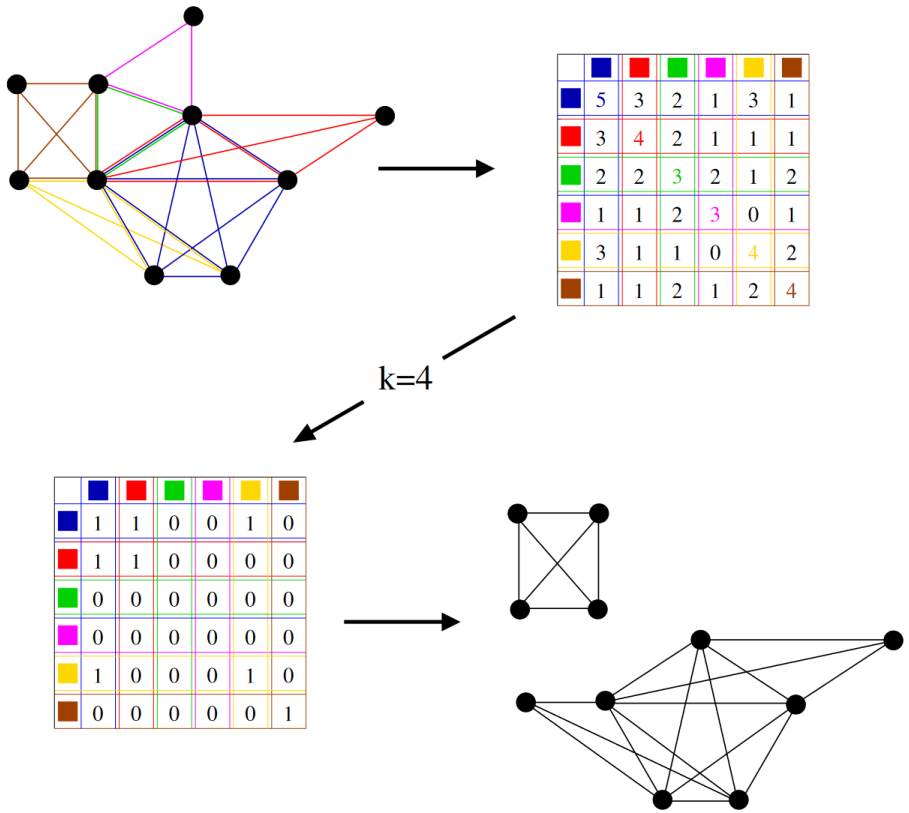


Figure: 4 clique communities: we delete the off-diagonal elements that are smaller than 3 and also the diagonal elements that are smaller than 4. The remaining connected components are the 4-clique-communities.

Clique percolation: association graphs

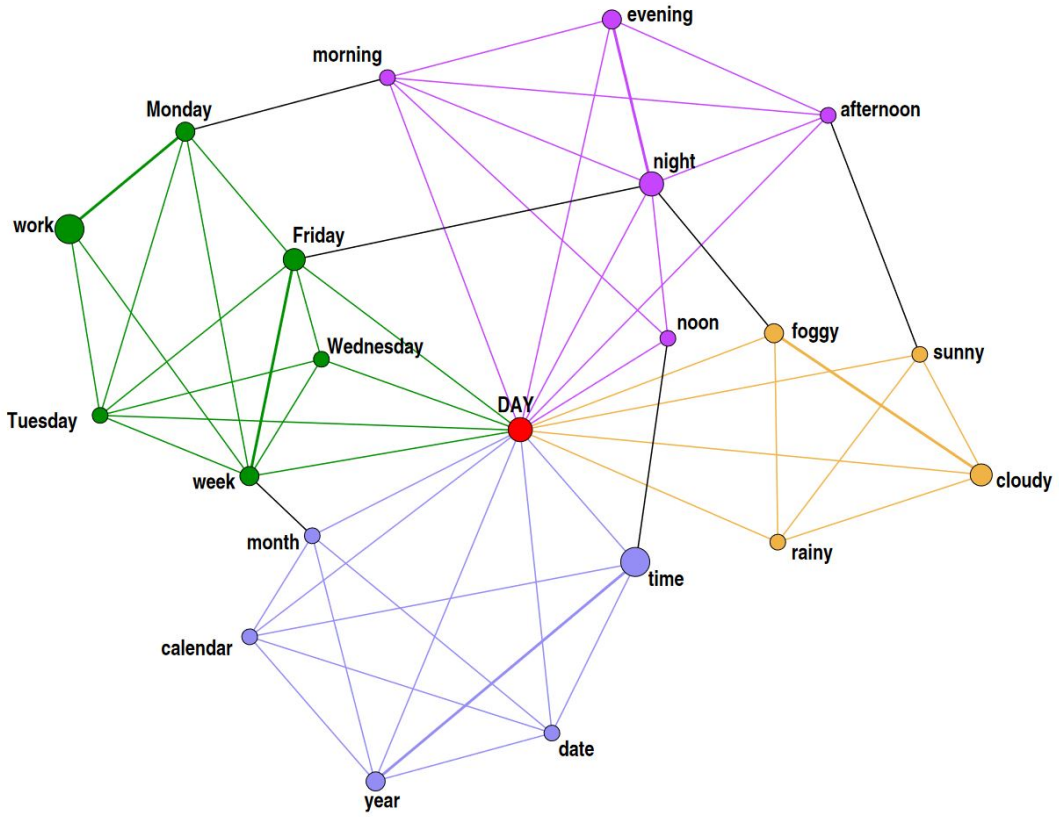


Figure: The green community is related to work-days, the purple one consists of day times, the yellow one can be associated to weather, whereas the blue community is related to the calendar.

Clique percolation: association graphs

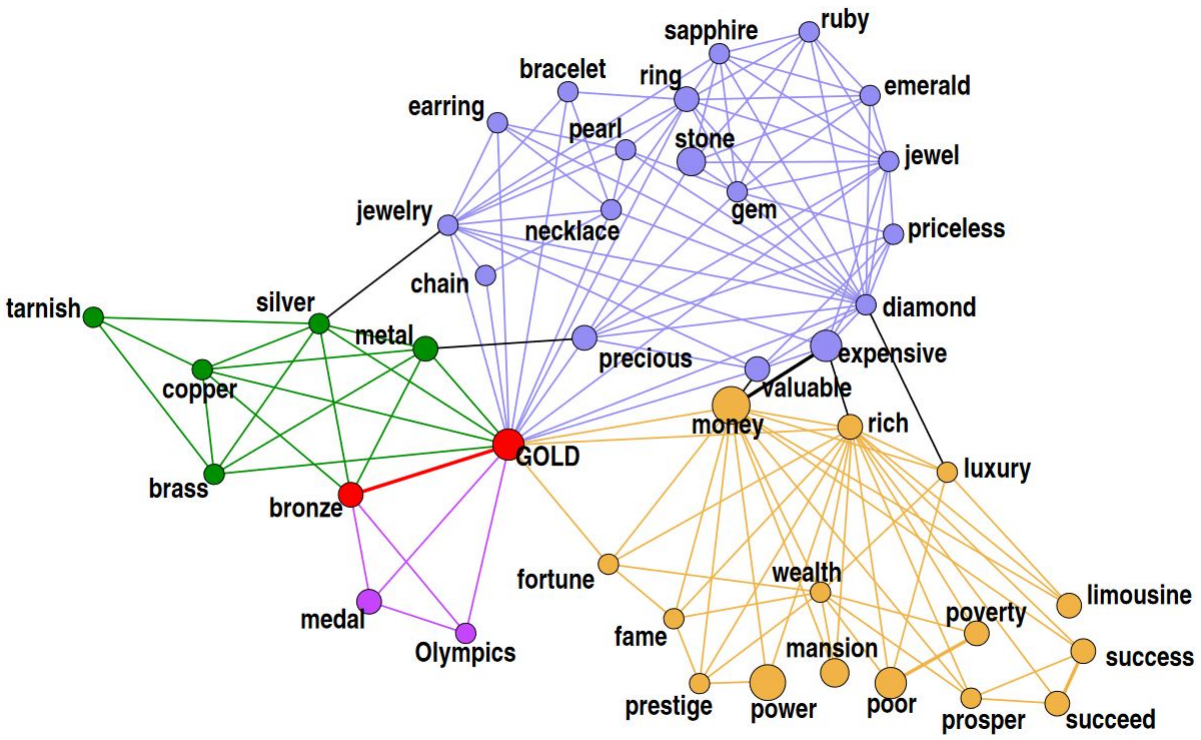


Figure: The purple community is related to Olympic medals, the green one consists of metals, the blue one can be associated to jewels and finally the yellow community is related to welfare.

Community detection based on modularity

Modularity

Modularity : a measure of how well a graph is partitioned into communities.

Given a partitioning of the graph into groups $C1, \ldots, Ck$:
M = $\sum_{i \in [1,k]}$ [(# edges within Ci) – (expected # edges within Ci)]

What is the **expected number of edges** ? We compute the number of edges in a null model (a type of random object that matches one specific object in some of its features, here the degree distribution).

Modularity - null model

Given a graph G with n nodes and m edges, construct a rewired graph G*, with the same degree distribution but random connections.

The expected number of edges between nodes n_i and n_j of degrees $deg(n_i)$ and $deg(n_i)$ equals to:

$$deg(n_i)deg(n_j)/2m$$

The rewired graph is called the null model or the configuration model.

Modularity

$$M = \sum_{n_i, n_j \in N} \frac{1}{2m} (a_{ij} - \frac{deg(n_i) deg(n_j)}{2m}) \delta(c_i, c_j)$$

where $m = |E|$,
a_{ij} = 1 if i is connected with j,
$\delta(c_i, c_j)$ = 1 if n_i and n_j belong to the same community, 0 otherwise

Modularity maximization: assigning nodes to communities to maximize modularity. Modularity values take range $[-1/2, 1]$, positive if the number of edges within groups exceeds the expected number. Modularity greater than 0.3 means we observe community structure. Modularity maximization is NP-hard.

Modularity maximization: Louvain algorithm

The Louvain algorithm has two phases that are repeated iteratively until merging the communities no longer improves the modularity.

Phase 1:

1. Each node in the network is assigned to its own community.
2. For each n_i, we remove n_i from its own community and move it into the community of a neighbor n_j.
3. Once we visited all neighbours, n_i is placed into the community that gives the greatest modularity increase. n_i can remain in its community.
4. We visit repeatedly and sequentially all nodes until no modularity increase can occur. Note that the algorithm chooses a random order of nodes.

Modularity maximization: Louvain algorithm

The Louvain algorithm has two phases that are repeated iteratively until merging the communities no longer improves the modularity.

Phase 2:

1. We build a new graph where the nodes are the communities from the previous phase.
2. Any links between nodes of the same community become self-loops and links from nodes in the same community to nodes in a different community are weighted edges between communities.
3. The first phase is re-applied.

Modularity maximization: Louvain algorithm

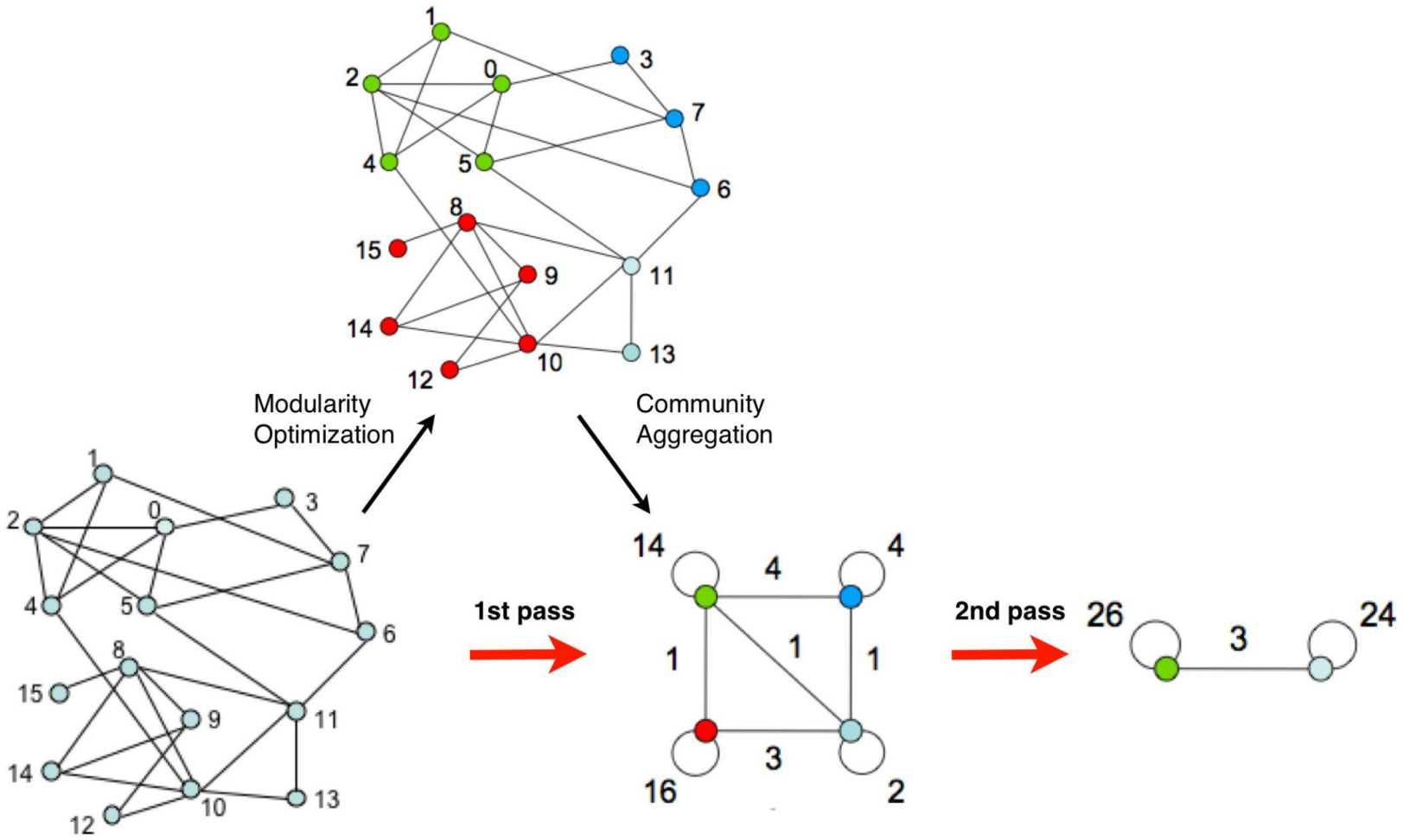


Figure: Example from [BGLL08]

Modularity: the resolution limit

Resolution limit: If a network is large enough, the expected number of edges between two groups of nodes in modularity's null model may be smaller than one. If this is the case, a single edge would make the groups of nodes join into a single community.
To alleviate the problem we introduce the parameter R:

$$M = \sum_{n_i, n_j \in N} \frac{1}{2m}(a_{ij} - R\frac{deg(n_i)deg(n_j)}{2m})\delta(c_i, c_j)$$

where R is the resolution parameter.
By default $R = 1$. When $R > 1$ the algorithm will give smaller communities, while for $R < 1$ it will give bigger communities.

Community detection based on betweenness centrality

Betweenness centrality

Betweenness centrality of the node $n \in G$ reflects how many of the shortest paths in G pass through n.

For $n_i, n_j \in N$, let $n_{i,j}$ be the number of distinct shortest paths from i to j. For $n_i, n_j, n_k \in N$, let $n^k_{i,j}$ be the number of shortest paths from n_i to n_j passing through n_k.

The betweenness centrality of n_k is:

$$b_k = \sum_{i,j \text{ such that } n_{i,j} > 0} (n^k_{i,j} / n_{i,j})$$

Betweenness centrality - for edges!

Betweenness centrality of the edge $e_k \in G$ reflects how many of the shortest paths in G pass through e_k.

For $n_i, n_j \in N$, let $n_{i,j}$ be the number of distinct shortest paths from i to j. For $n_i, n_j k \in N$, let $n^k_{i,j}$ be the number of shortest paths from n_i to n_j passing through e_k.

The betweenness centrality of e_k is:

$$b_k = \sum_{i,j \text{ such that } n_{i,j} > 0} (n^k_{i,j} / n_{i,j})$$

Girvan-Newman algorithm

Community detection using betweeness:

If a graph contains communities that are only loosely connected by a few intergroup edges, then all shortest paths between different communities must go along one of these few edges.

Thus, the edges connecting communities will have high edge betweenness. By removing these edges, we separate the groups and so reveal the underlying community structure of the graph.

Girvan-Newman algorithm

The betweenness of all existing edges in the network is computed.

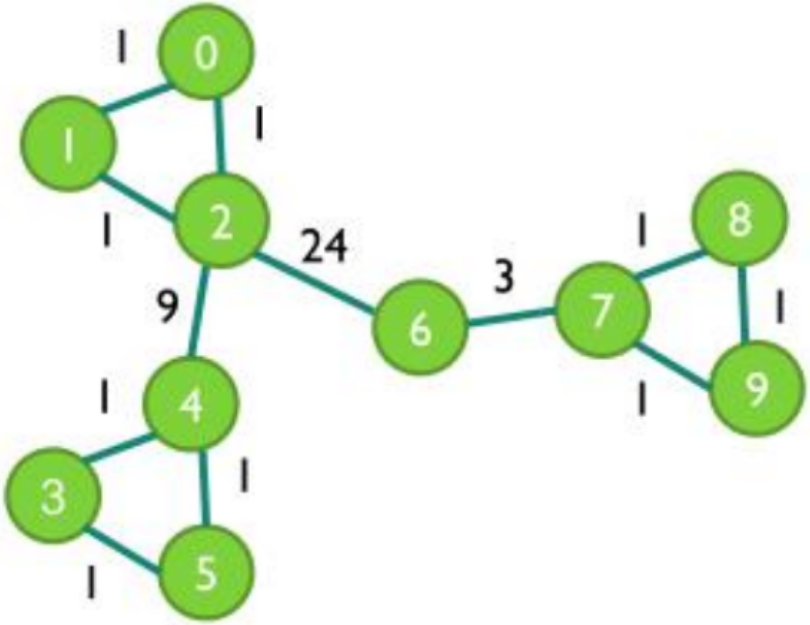
Then we iterate until we reached a stopping condition:

1. The edge(s) with the highest betweenness are removed.
2. The betweenness of all edges affected by the removal is recalculated.

Stopping condition:

- we run the algorithm until there are no more edges, we will get a dendrogram (representation of a tree);
- each time we have a new partition we compute the modularity and we take the best partition.

Girvan-Newman algorithm



As we move down the tree, we see the partitioning of groups.

DENDROGRAM

Evaluating Community Detection: Purity

- **Purity:** We map each computed community to the ground-truth community with the highest overlap to calculate the percentage of "correctly" clustered nodes [LCZ19].
- **Formula:**

$$Purity(B, A) = \frac{1}{n} \sum_{k} \max_{r} \left| B_k \cap A_r \right|$$

  - n is the total number of samples (nodes).
  - $B = \{B_1, B_2, \ldots, B_c\}$ are the detected communities.
  - $A = \{A_1, A_2, \ldots, A_{c^*}\}$ are the ground-truth communities.

- **Limitation:** The mapping is many-to-one rather than one-to-one [. This means multiple detected communities can be mapped to the same ground-truth community, which can artificially inflate the purity value to 1 and yield unreasonable results.

References I

Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre.
Fast unfolding of communities in large networks.
Journal of statistical mechanics: theory and experiment, 2008(10):P10008, 2008.

Sergey Brin and Larry Page.
The anatomy of a large-scale hypertextual web search engine.
Computer networks and ISDN systems, 1998.

Xin Liu, Hui-Min Cheng, and Zhong-Yuan Zhang.
Evaluation of community detection methods.
IEEE Transactions on Knowledge and Data Engineering, 32(9):1736–1746, 2019.

Wassily W. Leontief.
The Structure of American Economy, 1919-1929.
Harvard University Press, 1941.

John R. Seeley.
The net of reciprocal influence: A problem in treating sociometric data.
The Canadian Journal of Psychology, 1949.